

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA06

COURSE OUTCOME COVERED

CO 1: Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.

(BL-1, BL-2, BL-3)

Assessment Tool Weightage: 40%

QUESTIONS: 10

Annexure A

Application-Based Question

Total Marks: 50

Code of Conduct:

S.PRITHISH (192421134) *(Name/Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

***Signature of the
student: S.PRITHISH***

26.

SOURCE CODE

```
import streamlit as st
import numpy as np
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
import plotly.graph_objects as go
import math
import random

# -----
# Streamlit Page Configuration
# -----
st.set_page_config(
    page_title="Smart Grid Energy Distribution System",
    layout="wide"
)

st.title("⚡ Smart Grid Energy Distribution System")
st.markdown("---")

# -----
# Master Theorem Analysis
# -----
st.header("Master Theorem Analysis")

st.latex(r"T(n)=4T(n/2)+n\log n")

a = 4
b = 2

st.subheader("Step 1: Identify Parameters")
st.write("a =", a)
st.write("b =", b)

log_value = math.log(a, b)
```

```

st.latex(r"\log_b a = \log_2 4 = 2")

st.subheader("Step 2: Compare Functions")
st.latex(r"f(n)=n\log n")
st.latex(r"n^{\log_b a}=n^2")

st.write("Since")
st.latex(r"n\log n = O(n^{2-\epsilon}) ")

st.success("Master Theorem Case 1 Applies")

st.subheader("Time Complexity")
st.latex(r"T(n)=\Theta(n^2) ")

st.success("Overall Time Complexity =  $\Theta(n^2)$ ")

st.markdown("---")

# -----
# Smart Grid Network
# -----
st.header("Smart Grid Energy Distribution Network")

nodes = st.slider("Number of Energy Stations", 5, 30, 12)

# Create Random Tree (Compatible with all NetworkX versions)
G = nx.Graph()

for i in range(nodes):
    G.add_node(i)

for i in range(1, nodes):
    G.add_edge(i, random.randint(0, i - 1))

pos = nx.spring_layout(G, seed=42)

fig, ax = plt.subplots(figsize=(8, 6))

nx.draw_networkx_nodes(
    G,

```

```

        pos,
        node_color="orange",
        node_size=700,
        ax=ax
    )

    nx.draw_networkx_edges(
        G,
        pos,
        edge_color="green",
        width=2,
        ax=ax
    )

    nx.draw_networkx_labels(
        G,
        pos,
        font_size=10,
        font_weight="bold",
        ax=ax
    )

plt.title("Smart Grid Energy Distribution")
st.pyplot(fig)

st.markdown("---")

# -----
# Performance Table
# -----
st.header("Performance Analysis")

sizes = np.array([16, 32, 64, 128, 256, 512, 1024])

operations = sizes ** 2

df = pd.DataFrame({
    "Network Size": sizes,
    " $\Theta(n^2)$  Operations": operations
})

```

```

st.dataframe(df, use_container_width=True)

st.markdown("----")

# -----
# Complexity Graph
# -----
st.header("Time Complexity Graph")

fig1 = go.Figure()

fig1.add_trace(
    go.Scatter(
        x=sizes,
        y=operations,
        mode="lines+markers",
        name="Θ(n²) "
    )
)

fig1.update_layout(
    title="Master Theorem Complexity",
    axis_title="Network Size",
    axis_title="Operations",
    template="plotly_dark",
    height=500
)

st.plotly_chart(fig1, use_container_width=True)

st.markdown("----")

# -----
# Growth Comparison
# -----
st.header("Asymptotic Growth Comparison")

linear = sizes
nlogn = sizes * np.log2(sizes)

```

```

quadratic = sizes ** 2
cubic = sizes ** 3

fig2 = go.Figure()

fig2.add_trace(go.Scatter(x=sizes, y=linear,
                          mode="lines+markers",
                          name="O(n) "))

fig2.add_trace(go.Scatter(x=sizes, y=nlogn,
                          mode="lines+markers",
                          name="O(n log n)"))

fig2.add_trace(go.Scatter(x=sizes, y=quadratic,
                          mode="lines+markers",
                          name="Θ(n²) "))

fig2.add_trace(go.Scatter(x=sizes, y=cubic,
                          mode="lines+markers",
                          name="O(n³) "))

fig2.update_layout(
    title="Growth Rate Comparison",
    xaxis_title="Network Size",
    yaxis_title="Operations",
    template="plotly_dark",
    height=600
)

st.plotly_chart(fig2, use_container_width=True)

st.markdown("---")

# -----
# Summary Table
# -----

st.header("Asymptotic Notation")

summary = pd.DataFrame({
    "Notation": ["Big-O", "Big-Theta", "Big-Omega"],

```

```
    "Meaning": ["Upper Bound", "Tight Bound", "Lower Bound"],
    "Result": [" $O(n^2)$ ", " $\Theta(n^2)$ ", " $\Omega(n^2)$ "]
})
```

```
st.table(summary)
```

```
st.markdown("---")
```

```
# -----
```

```
# Final Interpretation
```

```
# -----
```

```
st.header("Interpretation")
```

```
st.info("""
```

Recurrence Relation:

$T(n) = 4T(n/2) + n \log n$

Parameters:

- $a = 4$
- $b = 2$
- $\log_2 4 = 2$

Since:

$n \log n = O(n^{2-\epsilon})$

Master Theorem Case 1 applies.

Therefore,

Time Complexity = $\Theta(n^2)$

Space Complexity = $O(\log n)$

Performance Analysis:

- Small Smart Grid → Very Fast
- Medium Smart Grid → Efficient

- Large Smart Grid → Quadratic Growth

The algorithm scales polynomially and is suitable for practical energy distribution systems.

```
"""
```

```
st.success("Application Executed Successfully ✔")
```


REPORT

SCREENSHOTS OF SOURCE CODE (APP.PY)

```
import streamlit as st
import numpy as np
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
import plotly.graph_objects as go
import math
import random

# -----
# Streamlit Page Configuration
# -----
st.set_page_config(
    page_title="Smart Grid Energy Distribution System",
    layout="wide"
)

st.title("⚡ Smart Grid Energy Distribution System")
st.markdown("---")

# -----
# Master Theorem Analysis
# -----
st.header("Master Theorem Analysis")

st.latex(r"T(n)=4T(n/2)+n\log n")

a = 4
b = 2

st.subheader("Step 1: Identify Parameters")
st.write("a =", a)
st.write("b =", b)
```

```

log_value = math.log(a, b)

st.latex(r"\log_b a = \log_2 4 = 2")

st.subheader("Step 2: Compare Functions")
st.latex(r"f(n)=n\log n")
st.latex(r"n^{\log_b a}=n^2")

st.write("Since")
st.latex(r"n\log n = O(n^{2-\epsilon}) ")

st.success("Master Theorem Case 1 Applies")

st.subheader("Time Complexity")
st.latex(r"T(n)=\Theta(n^2) ")

st.success("Overall Time Complexity =  $\Theta(n^2)$ ")

st.markdown("----")

# -----
# Smart Grid Network
# -----
st.header("Smart Grid Energy Distribution Network")

nodes = st.slider("Number of Energy Stations", 5, 30, 12)

# Create Random Tree (Compatible with all NetworkX versions)
G = nx.Graph()

for i in range(nodes):
    G.add_node(i)

for i in range(1, nodes):
    G.add_edge(i, random.randint(0, i - 1))

pos = nx.spring_layout(G, seed=42)

fig, ax = plt.subplots(figsize=(8, 6))

```

```

nx.draw_networkx_nodes(
    G,
    pos,
    node_color="orange",
    node_size=700,
    ax=ax
)

nx.draw_networkx_edges(
    G,
    pos,
    edge_color="green",
    width=2,
    ax=ax
)

nx.draw_networkx_labels(
    G,
    pos,
    font_size=10,
    font_weight="bold",
    ax=ax
)

plt.title("Smart Grid Energy Distribution")
st.pyplot(fig)

st.markdown("---")

# -----
# Performance Table
# -----
st.header("Performance Analysis")

sizes = np.array([16, 32, 64, 128, 256, 512, 1024])

operations = sizes ** 2

df = pd.DataFrame({
    "Network Size": sizes,

```

```

        "Θ(n²) Operations": operations
    })

st.dataframe(df, use_container_width=True)

st.markdown("---")

# -----
# Complexity Graph
# -----
st.header("Time Complexity Graph")

fig1 = go.Figure()

fig1.add_trace(
    go.Scatter(
        x=sizes,
        y=operations,
        mode="lines+markers",
        name="Θ(n²) "
    )
)

fig1.update_layout(
    title="Master Theorem Complexity",
    xaxis_title="Network Size",
    yaxis_title="Operations",
    template="plotly_dark",
    height=500
)

st.plotly_chart(fig1, use_container_width=True)

st.markdown("---")

# -----
# Growth Comparison
# -----
st.header("Asymptotic Growth Comparison")

```

```

linear = sizes
nlogn = sizes * np.log2(sizes)
quadratic = sizes ** 2
cubic = sizes ** 3

fig2 = go.Figure()

fig2.add_trace(go.Scatter(x=sizes, y=linear,
                          mode="lines+markers",
                          name="O(n) "))

fig2.add_trace(go.Scatter(x=sizes, y=nlogn,
                          mode="lines+markers",
                          name="O(n log n) "))

fig2.add_trace(go.Scatter(x=sizes, y=quadratic,
                          mode="lines+markers",
                          name="Θ(n²) "))

fig2.add_trace(go.Scatter(x=sizes, y=cubic,
                          mode="lines+markers",
                          name="O(n³) "))

fig2.update_layout(
    title="Growth Rate Comparison",
    xaxis_title="Network Size",
    yaxis_title="Operations",
    template="plotly_dark",
    height=600
)

st.plotly_chart(fig2, use_container_width=True)

st.markdown("---")

# -----
# Summary Table
# -----

st.header("Asymptotic Notation")

```

```
summary = pd.DataFrame({
    "Notation": ["Big-O", "Big-Theta", "Big-Omega"],
    "Meaning": ["Upper Bound", "Tight Bound", "Lower Bound"],
    "Result": ["O(n^2)", "Θ(n^2)", "Ω(n^2)"]
})
```

```
st.table(summary)
```

```
st.markdown("---")
```

```
# -----
```

```
# Final Interpretation
```

```
# -----
```

```
st.header("Interpretation")
```

```
st.info("""
```

Recurrence Relation:

$T(n) = 4T(n/2) + n \log n$

Parameters:

- $a = 4$
- $b = 2$
- $\log_2 4 = 2$

Since:

$n \log n = O(n^{2-\epsilon})$

Master Theorem Case 1 applies.

Therefore,

Time Complexity = $\Theta(n^2)$

Space Complexity = $O(\log n)$

Performance Analysis:

- Small Smart Grid → Very Fast

- Medium Smart Grid → Efficient
- Large Smart Grid → Quadratic Growth

The algorithm scales polynomially and is suitable for practical energy distribution systems.

```
""")
```

```
st.success("Application Executed Successfully ✔")
```

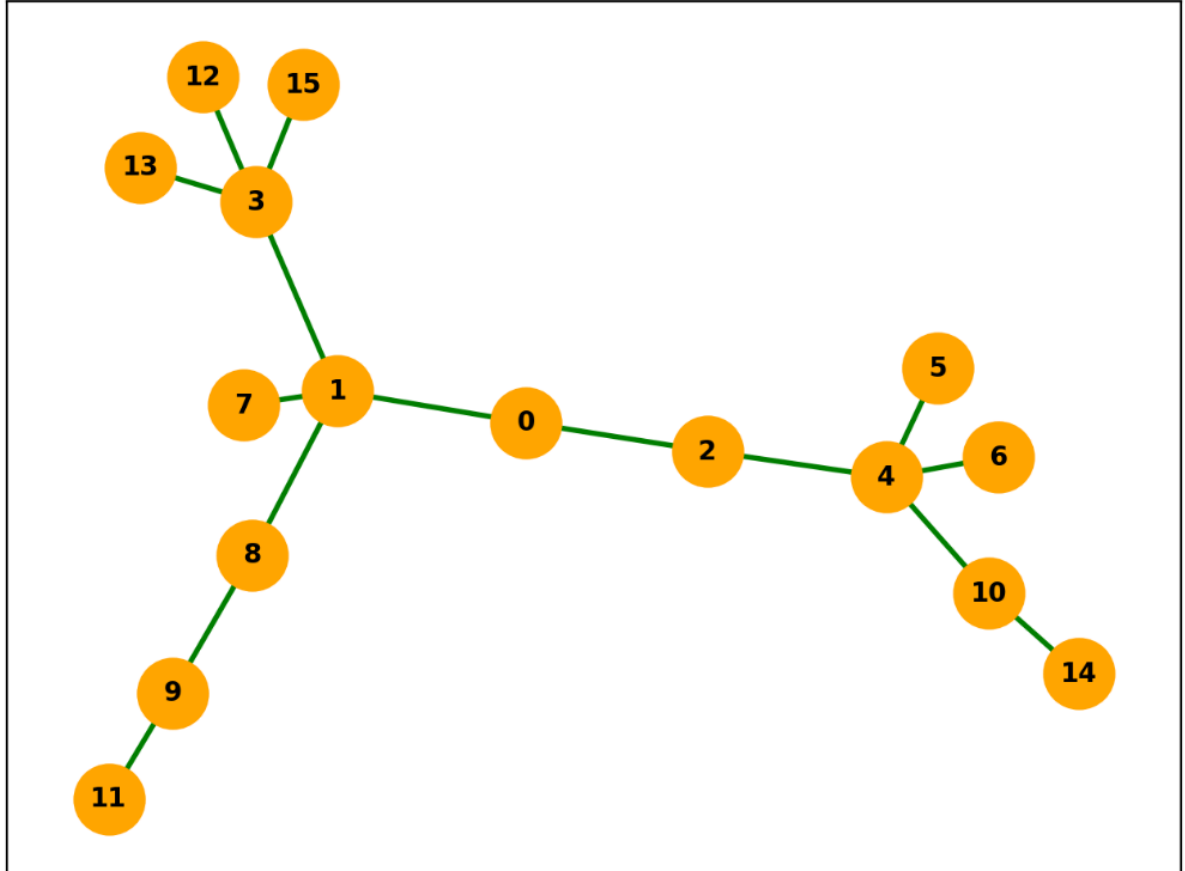
OUTPUT

Smart Grid Energy Distribution System

Master Theorem Analysis

$$T(n) = 4T(n/2) + n \log n$$

Smart Grid Energy Distribution



ANALYSIS

